

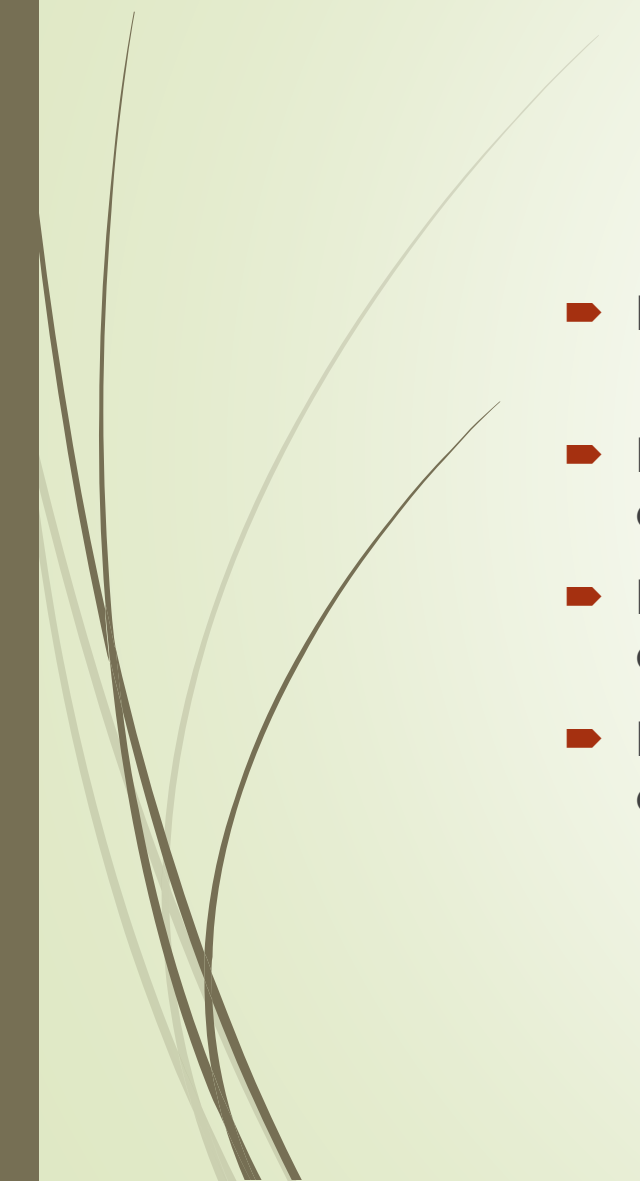


PATRÓN FLYWEIGHT

Ing. Zoila Ruiz, PhD



Patrón Flyweight

- Patrón estructural, que buscan solucionar problemas de composición (agregación) de clases y objetos.
 - Busca eliminar o reducir la redundancia cuando se tiene una gran cantidad de objetos que comparten mucha información
 - Este patrón se utiliza para aumentar el rendimiento de aplicaciones en las que se usan multitud de objetos similares.
 - El objetivo de este patrón es eliminar la redundancia existente mediante la compartición de las características comunes.
- 



Patrón Flyweight

Objetivos:

- Utilizar uso compartido de propiedades entre objetos para mejorar la eficiencia del sistema
- Eliminar o reducir la redundancia cuando tenemos una gran cantidad de objetos que contienen información idéntica, además de lograr equilibrio entre flexibilidad y rendimiento.



Patrón Flyweight

Se utiliza cuando :

- Cuando se necesita eliminar o reducir la redundancia
- La memoria es crítica para el rendimiento de la aplicación
- La identidad de los objetos es irrelevante
- Es ampliamente utilizado en programación gráfica

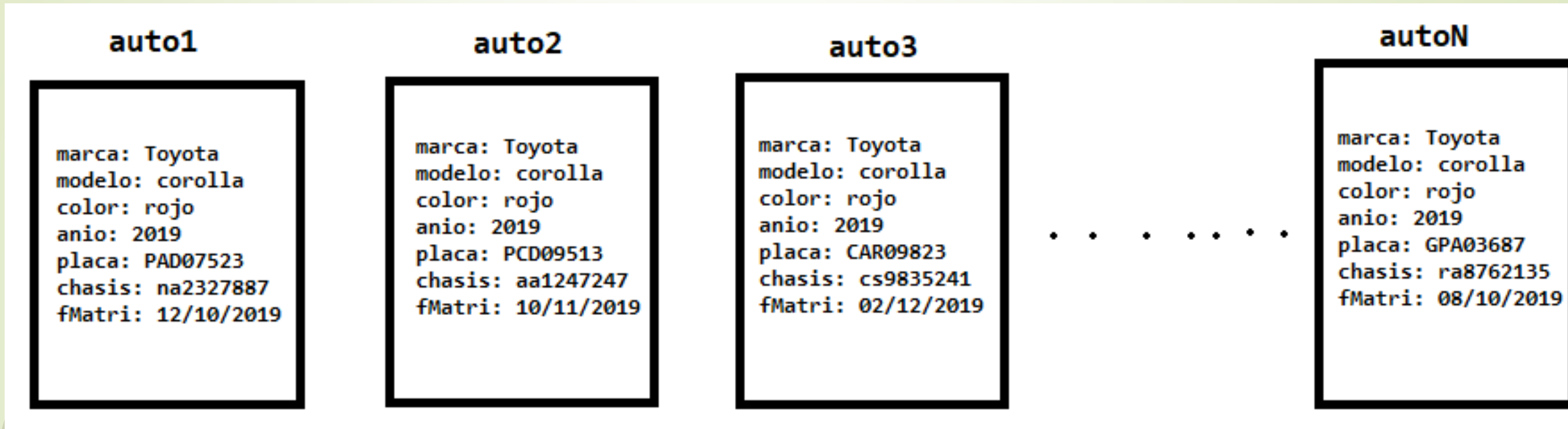


Patrón Flyweight

Elementos que debemos identificar para implementar el patrón:

- Componentes intrínsecos: Son los datos compartidos por todos los objetos de un subtipo determinado
- Componentes extrínsecos: Se calculan fuera del objeto Flyweight.

Patrón Flyweight

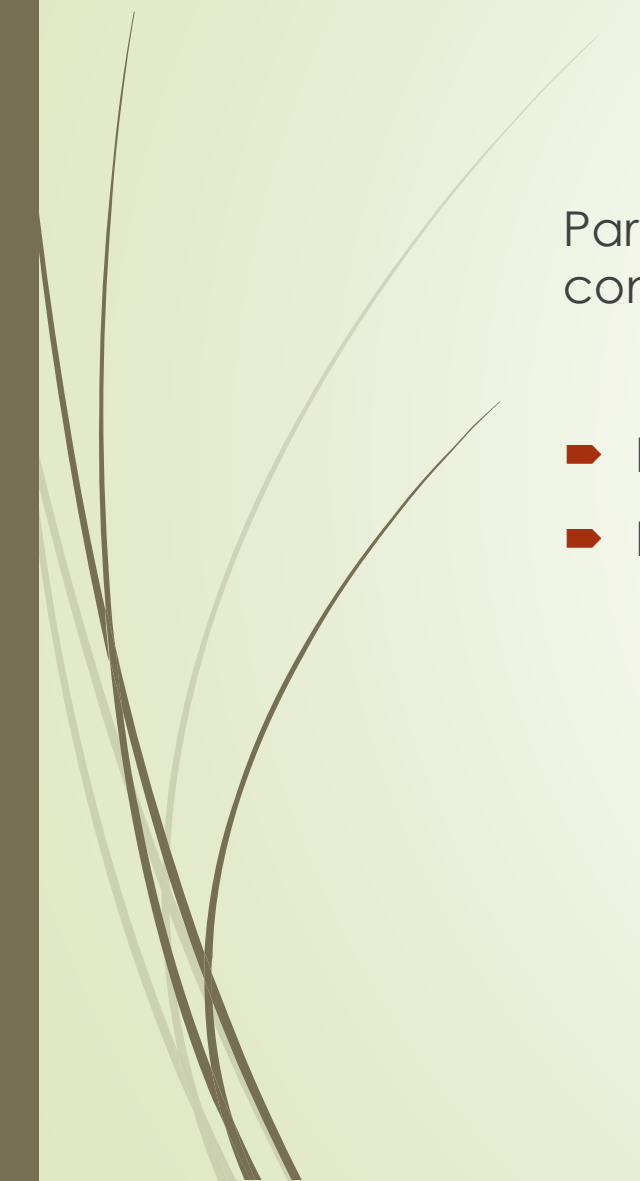


- Vemos que los objetos comparten características comunes, es decir sus atributos tienen el mismo valor. Estos serían los elementos intrínsecos y los atributos cuyos valores son distintos, serían los elementos extrínsecos.

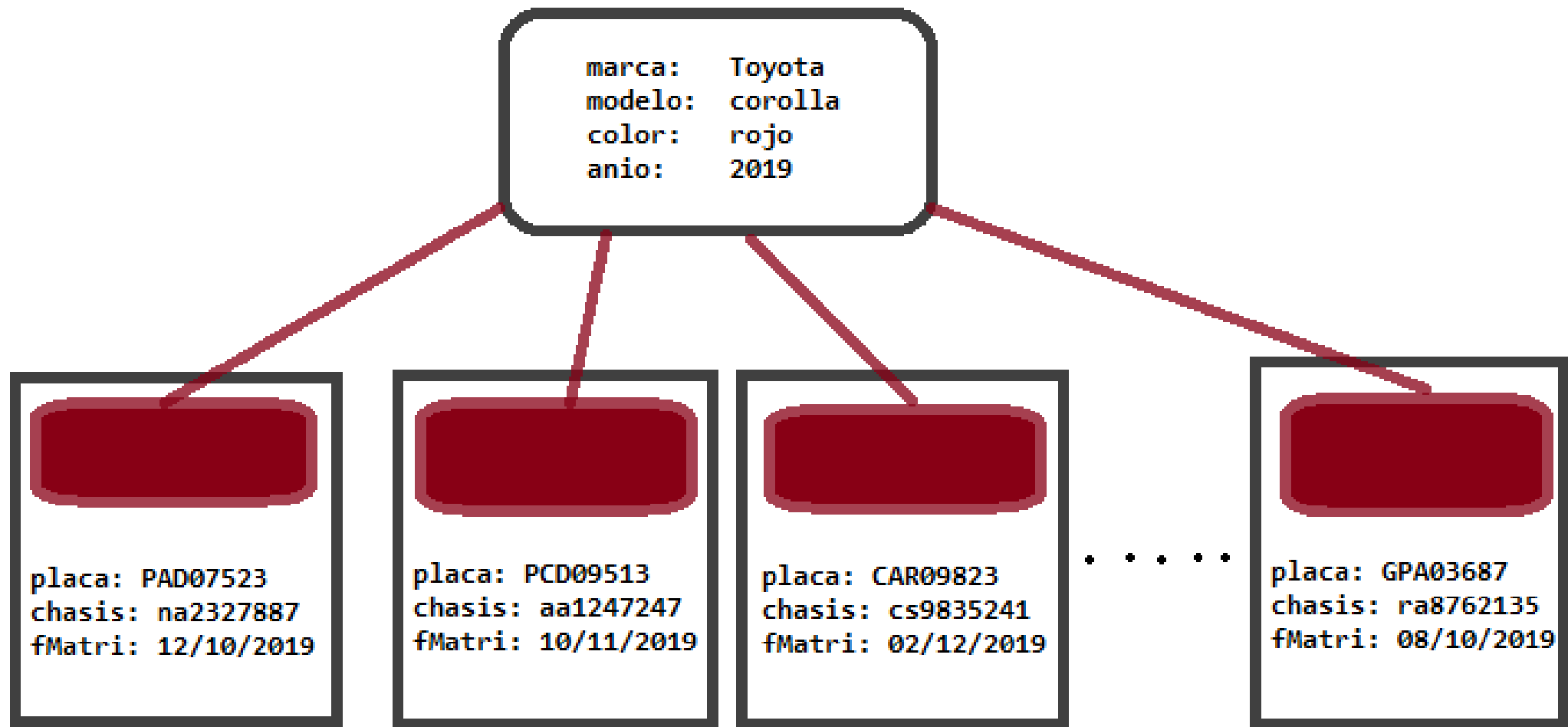


Patrón Flyweight

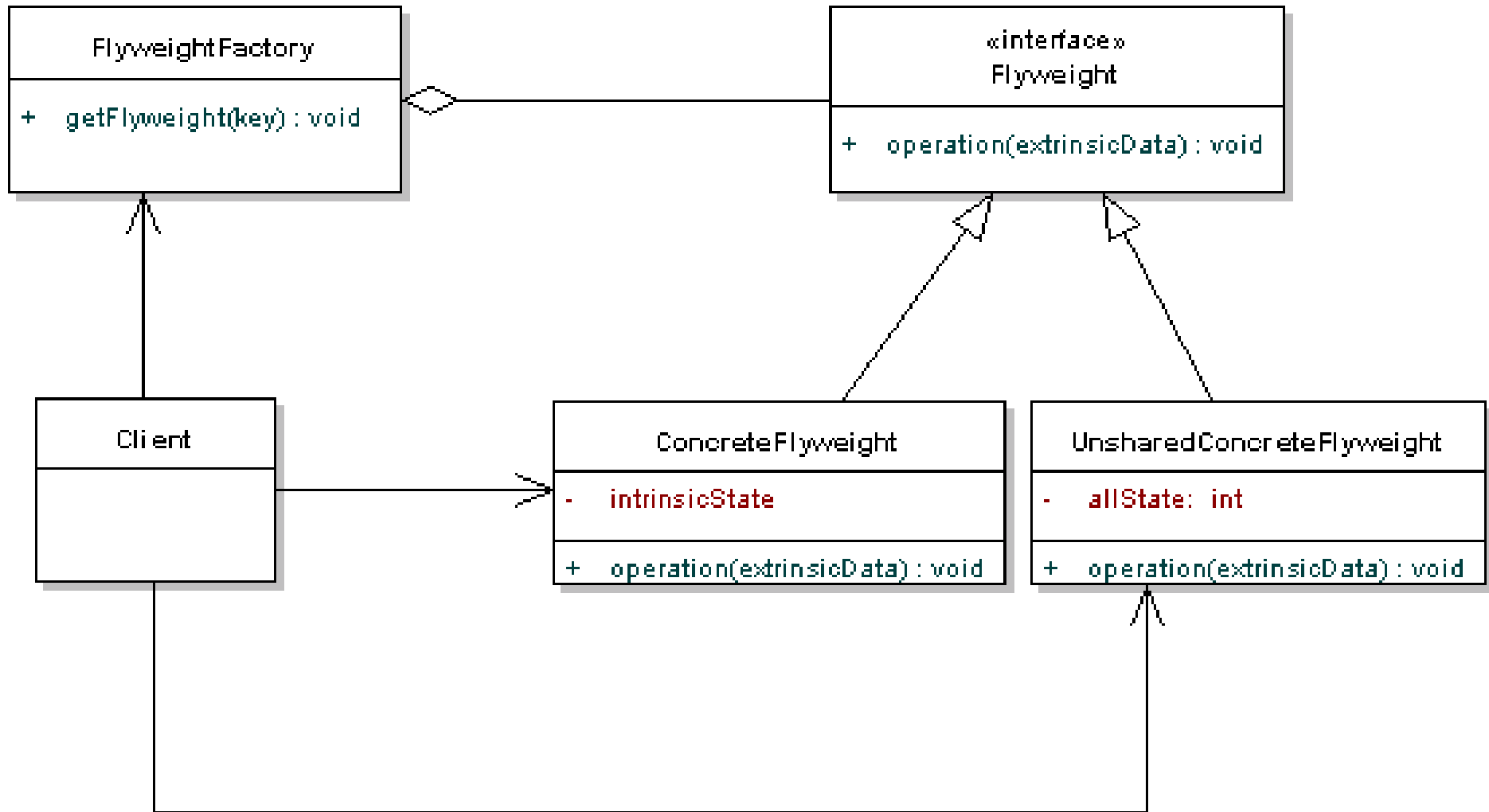
Para aplicar este patrón debemos empezar por dividir los elementos que conforman el objeto en estados:

- ▀ Estado intrínseco.- elementos que tengan en común
 - ▀ Estado extrínseco.- elementos particulares a cada tipo
- 

Patrón Flyweight



Patrón Flyweight





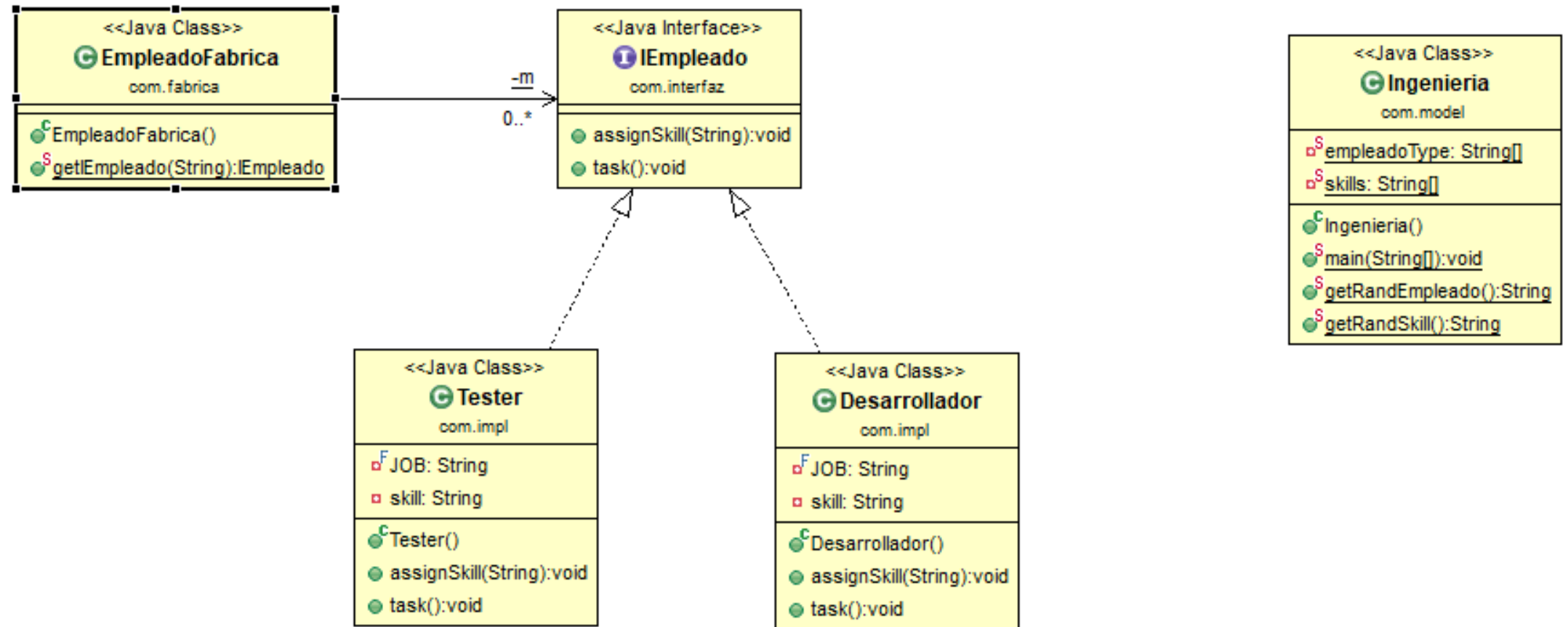
Patrón Flyweight

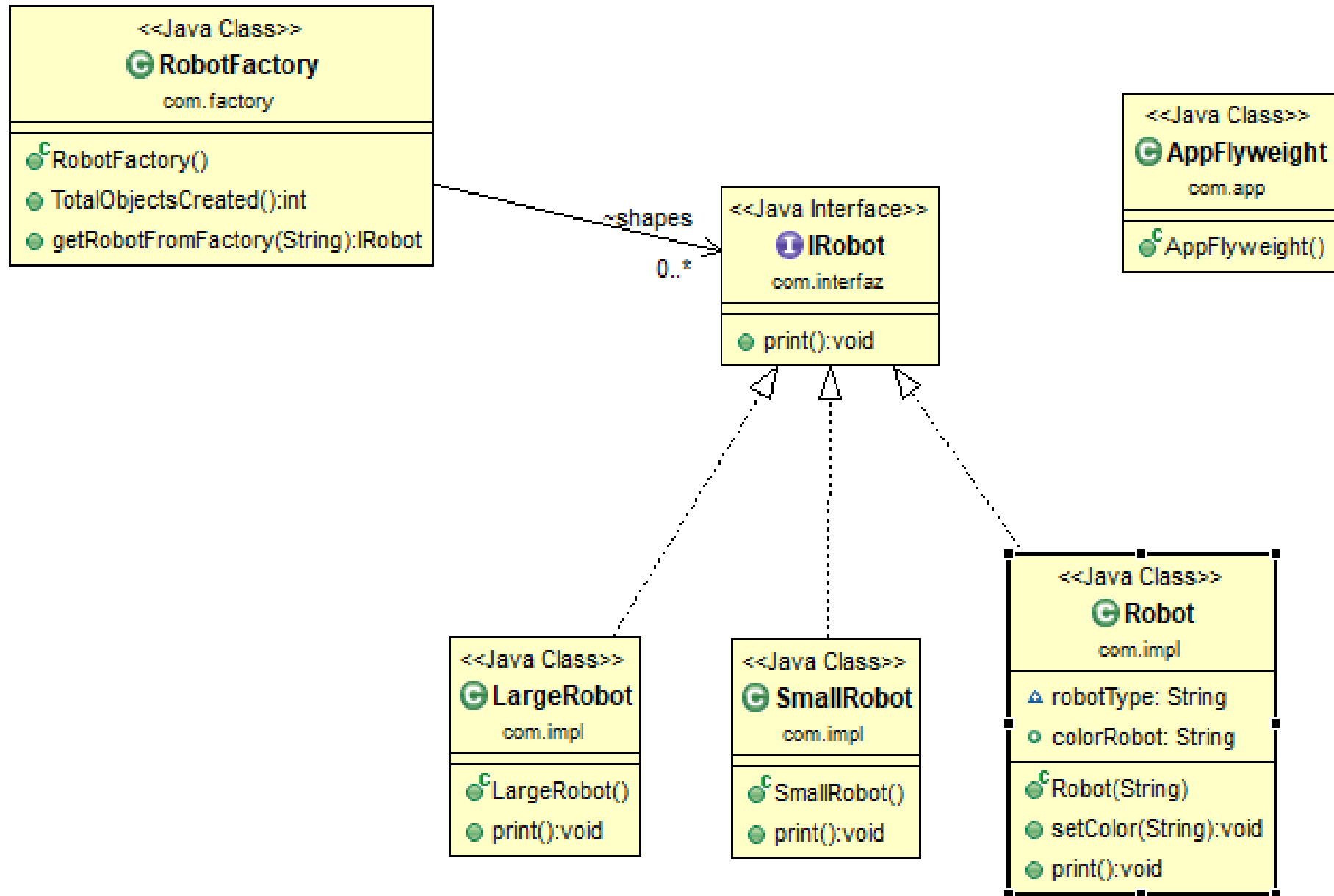
El patrón flyweight está ligado con el patrón factory, pues permite que el objeto sea quien gestione la separación entre la parte común (intrínseca) y la parte privada (extrínseca), centralizando el proceso.

Elementos:

- **Flyweight:** declara la interfaz a través de la cual los flyweights pueden recibir y actuar sobre los estados no compartidos.
- **ConcreteFlyweight:** implementa la interfaz Flyweight y almacena los estados compartidos.
- **UnsharedConcreteFlyweight:** no todas las subclases tienen por qué ser compartidas.
- **FlyweightFactory:** crea y gestiona los objetos flyweight. Proporciona una instancia existente o crea una.
- **Client:** contiene referencias a los flyweights, calcula o almacena los estados no compartidos.

Patrón Flyweight





Patrón Flyweight

